COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE-010	**SECTION:**	CPE21S2
DATE PERFORMED:	07/29/2026	**DATE SUBMITTED:**	08/05/2026
NAME:	Julian Benedict D.R. Barizo	**INSTRUCTOR:**	Jimlord Quejado

ACTIVITY NO. ___ [ACTIVITY NAME]

1. PROCEDURE OUTPUT

```cpp
#include <iostream>
#include <stack>

int main() {

    std::stack<int> stack1;
    std::cout<<"Testing the stack STL"<<std::endl;


    //isEmpty
    std::cout<<"Is the stack empty? " << stack1.empty() << std::endl;


    //push
    stack1.push(10);
    std::cout<< "the top of the stack is: " << stack1.top() << std::endl;
    stack1.push(9);
    std::cout<< "the top of the stack is: " << stack1.top() << std::endl;
    stack1.push(8);
    std::cout<< "the top of the stack is: " << stack1.top() << std::endl;
    stack1.push(7);
    std::cout<< "the top of the stack is: " << stack1.top() << std::endl;
    stack1.push(6);
    std::cout<< "the top of the stack is: " << stack1.top() << std::endl;

    //pop
    stack1.pop();
    std::cout<< "the top of the stack is: " << stack1.top() << std::endl;

    std::cout<<"The size of the stack is: " << stack1.size() << std::endl;

    std::cout<<"is the stack empty? " <<stack1.empty()<<std::endl;
```

```cpp
//Experimenting with emplace
std::cout << "--Experimenting with the Emplace command--" << std::endl;


stack1.emplace(11);
std::cout<<"The Top of the stack after the emplacement is: " << stack1.top() << std::endl;

stack1.emplace(12);
std::cout<<"The Top of the stack after the emplacement is: " << stack1.top() << std::endl;
```

```cpp
//Experinmenting with Swap
    //Initialize a new stack for the swap command

std::stack<int> stack2;

//push elements in stack 2
stack2.push(67);
stack2.push(69);

 std::cout << "Before swap:" << std::endl;
 std::cout << "  stack1 top: " << stack1.top() << std::endl;
 std::cout << "  stack2 top: " << stack2.top() << std::endl;

//swap command
stack1.swap(stack2);

std::cout << "After swap:" << std::endl;
std::cout << "  stack1 top: " << stack1.top() << std::endl;
std::cout << "  stack2 top: " << stack2.top() << std::endl;
```

Code

```
Testing the stack STL
Is the stack empty? 1
the top of the stack is: 10
the top of the stack is: 9
the top of the stack is: 8
the top of the stack is: 7
the top of the stack is: 6
the top of the stack is: 7
The size of the stack is: 4
is the stack empty? 0
--Experimenting with the Emplace command--
The Top of the stack after the emplacement is: 11
The Top of the stack after the emplacement is: 12
Before swap:
  stack1 top: 12
  stack2 top: 69
After swap:
  stack1 top: 69
  stack2 top: 12
```

Output

For this part of the activity, I learnt about the various commands from the standard C++ stack library like push, pop, empty, top, and size and what their functions are. Secondly, we were also tasked to figure out how the emplace and the swap command works so I managed to experiment with the code and figured out that emplace places an element on the very top of the stack, and swap swaps the elements of a stack with another stack.

```cpp
#include <iostream>

//global declarations
#define maxCap 100
const size_t arrSize = 100;

int stackArr[maxCap];
int top = -1, newData;
//prototype functions
void push();
void pop();
void Top();
bool isEmpty();
bool isFull();
void displayElements();
```

```cpp
int main () {
//main driver
    int choice;

    while(true)
    {
        std::cout<< "================================================="<<std::endl;
        std::cout<<"Stack Operations \n";
        std::cout<<"1. PUSH 2. POP 3. TOP 4. is Empty 5. isFull 6. Display Elements" <<std::endl;
        std::cout<< "================================================="<<std::endl;
        std::cin>>choice;

        switch(choice){
            case 1: push();
            break;
            case 2: pop();
            break;
            case 3: Top();
            break;
            case 4: std::cout<<"is stack empty? " << isEmpty() << std::endl;
            break;
            case 5: std::cout<<"Is the stack full?" << isFull()<< std::endl;
            break;
            case 6: displayElements();
            break;
            default: std::cout<<"Invalid choice" << std::endl;
            break;
        }
    }


}
```

```cpp
//function definition
bool isEmpty() {
    //check the top if it is -1
    if (top == -1)
    return true;
    return false;
}

bool isFull() {
    if(top == maxCap - 1)return true;

    return false;
}

void push() {
    //error checking ??
    if (isFull()){
        std::cout<<"Stack overflow"<<std::endl;
        return;
    }
    //pushing to the stack
    std::cout<< "Enter a new Value: " << std::endl;
    std::cin >> newData;
    //how do we insert the data into the stack?
    stackArr[++top] = newData;
}
```

```
void pop(){
    //error checking
    if(isEmpty()) {
        std::cout<<"Stack underflow" << std::endl;
        return;
    };
    //Display the value that we are going to pop:
    std::cout<<"Popping: "<<stackArr[top]<<std::endl;


    //decrement the top value from the stack
    top--;
}

void Top(){
    //error catching
    if(isEmpty()){
        std::cout<<"The stack is empty" << std::endl;
        return;
    }
    //check the top value:
    std::cout<<stackArr[top]<<std::endl;



}

void displayElements(){
    //error checking
    if(isEmpty()) {
        std::cout <<"The stack is empty" << std::endl;
    }
    //for loop to iterate through the stack
    std::cout<<"The elements in the stack are: " << std::endl;
    for (int i = top; i >= 0; i--) {
        std::cout << stackArr[i] << " ";
    }
}
```

Code

```
================================================
Stack Operations
1. PUSH 2. POP 3. TOP 4. is Empty 5. isFull 6. Display Elements
================================================
1
Enter a new Value:
1
================================================
Stack Operations
1. PUSH 2. POP 3. TOP 4. is Empty 5. isFull 6. Display Elements
================================================
1
Enter a new Value:
2
================================================
Stack Operations
1. PUSH 2. POP 3. TOP 4. is Empty 5. isFull 6. Display Elements
================================================
1
Enter a new Value:
3
================================================
Stack Operations
1. PUSH 2. POP 3. TOP 4. is Empty 5. isFull 6. Display Elements
================================================
1
Enter a new Value:
4
================================================
Stack Operations
1. PUSH 2. POP 3. TOP 4. is Empty 5. isFull 6. Display Elements
================================================
1
Enter a new Value:
5
================================================
Stack Operations
1. PUSH 2. POP 3. TOP 4. is Empty 5. isFull 6. Display Elements
================================================
1
Enter a new Value:
6
```

```
==================================================
Stack Operations
1. PUSH 2. POP 3. TOP 4. is Empty 5. isFull 6. Display Elements
==================================================
1
Enter a new Value:
7
==================================================
Stack Operations
1. PUSH 2. POP 3. TOP 4. is Empty 5. isFull 6. Display Elements
==================================================
2
Popping: 7
==================================================
Stack Operations
1. PUSH 2. POP 3. TOP 4. is Empty 5. isFull 6. Display Elements
==================================================
3
6
==================================================
Stack Operations
1. PUSH 2. POP 3. TOP 4. is Empty 5. isFull 6. Display Elements
==================================================
6
The elements in the stack are:
6
5
4
3
2
1
==================================================
Stack Operations
1. PUSH 2. POP 3. TOP 4. is Empty 5. isFull 6. Display Elements
==================================================
```

Output

For this part of the activity, we tackled about stack implementation using arrays. We didn't use the stack from the C++ standard library this time and instead made our own functions. We initialized the similar functions from the library like push, pop, top, is empty, and is full and each of them did their own function within their respective category. Lastly, we initialized a switch case for user input so the user can choose which function they want to execute and what elements they want to put within the stack themselves. Lastly, we were tasked to make a function to display all the elements inside the stack and I did that by using a for loop to iterate throughout the stack by decrementing the top element one by one until it prints each and every one of the elements.

```cpp
#ifndef STACKLIST_H
#define STACKLIST_H

#include <iostream>

template <typename T>
class Node {
    public:
    T data; // VALUE WE NEED TO STORE
    Node* next; // POINTER TO THE NEXT NODE
};

// GLOBAL POINTER FOR THE TOP OF THE STACK
template <typename T>
Node<T>* head = nullptr;

// OPERATTIONS FOR THE STACK
```

```cpp
// PUSH
template <typename T>
void push(T newData) {
    // CREATE A NEW NODE AND ASSIGN THE newData
    Node<T>* newNode = new Node<T>();
    newNode->data = newData;

    // NEW NODE POINTS TO CURRENT TOP, THEN BECOMES THE NEW TOP
    newNode->next = head<T>;
    head<T> = newNode;
}
```

push(T newData) — Adds a new element to the top of the stack. Creates a new node holding newData, points it at the current head, then makes it the new head.

```cpp
// POP

template <typename T>
T pop() {
    // CHECK IF THE STACK IS EMPTY:
    if (head<T> == nullptr) {
        std::cerr << "Stack Underflow: Cannot pop from an empty stack." << std::endl;
        return T(); // RETURN A DEFAULT VALUE INSTEAD OF DEREFERENCING A NULL POINTER
    }

    // SAVE THE CURRENT TOP NODE
    Node<T>* temp = head<T>;
    // STORE ITS DATA
    T tempVal = temp->data;

    // MOVE THE TOP POINTER TO THE NEXT NODE
    head<T> = head<T>->next;

    // FREE THE REMOVED NODE
    delete temp;

    // RETURN THE REMOVED VALUE
    return tempVal;
}
```

pop() — Removes and returns the top element. Saves the top node's value, advances head to the next node, frees the old node, and returns the saved value. On underflow (empty stack), it now prints a warning and returns a default-constructed value instead of crashing.

```cpp
// isEmpty
template <typename T>
bool isEmpty() {
    return head<T> == nullptr;
}

// Top
template <typename T>
void Top() {
    // ERROR CATCHING:
    if (head<T> == nullptr) {
        std::cout << "Stack is empty. No top element." << std::endl;
        return;
    }
    // DISPLAY THE TOP ELEMENT
    std::cout << "Top element: " << head<T>->data << std::endl;
}
```

isEmpty() — Returns true if head is nullptr, false otherwise. Used as the loop condition to pop everything off the stack.

```cpp
// Top
template <typename T>
void Top() {
    // ERROR CATCHING:
    if (head<T> == nullptr) {
        std::cout << "Stack is empty. No top element." << std::endl;
        return;
    }
    // DISPLAY THE TOP ELEMENT
    std::cout << "Top element: " << head<T>->data << std::endl;
}
```

Top() — Peeks at the top element without removing it, printing its value (or an "empty" message).

```cpp
// DISPLAY ALL
template <typename T>
void displayALL() {
    // ERROR CATCHING:
    if (head<T> == nullptr) {
        std::cout << "Stack Underflow\n" << std::endl;
        return;
    }

    // TRAVERSE THE STACK AND DISPLAY ALL ELEMENTS
    Node<T>* temp = head<T>;
    std::cout << "Stack elements: \n";
    while (temp != nullptr) {
        std::cout << temp->data << " \n";
        temp = temp->next;
    }
    std::cout << std::endl;
}

#endif // STACKLIST_H
```

displayALL() — Traverses the stack from head to the end, printing every element (top to bottom). This is the "display all elements" function the task asked for — it's already implemented in stackList.h.

```cpp
#include <iostream>
#include "stackList.h"

int main() {

    std::cout << "--- PUSHING 'C' 'P' 'E' ONTO THE STACK ---\n";

    //PUSHING
    push('C');
    push('P');
    push('E');

    //DISPLAY THE CURRENT TOP

    Top<char>; /* TOP ELEMENT SHOULD BE 'E' */
    displayALL<char>();
    std::cout << "\n --- POPPIING ALL ELEMENT ---\n";

    while(!isEmpty<char>()) {
        std::cout<<"Popped: "<< pop<char>() <<std::endl;
    }

    std::cout << "\n --- TESTING THE UNDERFLOW CONDITION --- \n";
    pop<char>(); /* UNDERFLOW CONDITION */

    return 0;
}
```

Main CPP Code

```
--- PUSHING 'C' 'P' 'E' ONTO THE STACK ---
Stack elements:
E
P
C


 --- POPPIING ALL ELEMENT ---
Popped: E
Popped: P
Popped: C

 --- TESTING THE UNDERFLOW CONDITION ---
Stack Underflow: Cannot pop from an empty stack.
```

Output

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Stacks Using Arrays

```
#include <iostream>
#include <string>
using namespace std;

// ---------- STACK USING ARRAYS ----------
class ArrayStack {
private:
    char* arr;
    int capacity;
    int topIndex;

public:
    ArrayStack(int cap = 100) {
        capacity = cap;
        arr = new char[capacity];
        topIndex = -1; // -1 means empty
    }

    ~ArrayStack() {
        delete[] arr;
    }

    bool isEmpty() {
        return topIndex == -1;
    }

    bool isFull() {
        return topIndex == capacity - 1;
    }
```

```
    void push(char c) {
        if (isFull()) {
            cout << "Stack Overflow." << endl;
            return;
        }
        arr[++topIndex] = c;
    }

    char pop() {
        if (isEmpty()) {
            cout << "Stack Underflow." << endl;
            return '\0';
        }
        return arr[topIndex--];
    }
};

// ---------- SYMBOL HELPERS ----------
bool isOpening(char c) {
    return c == '(' || c == '{' || c == '[';
}

bool isClosing(char c) {
    return c == ')' || c == '}' || c == ']';
}

bool isMatchingPair(char open, char close) {
    return (open == '(' && close == ')') ||
           (open == '{' && close == '}') ||
           (open == '[' && close == ']');
}

// ---------- BALANCE CHECKING ALGORITHM ----------
bool isBalancedArray(const string& expr) {
    ArrayStack stack(expr.length() + 1);

    for (size_t i = 0; i < expr.length(); i++) {
        char c = expr[i];
```

```
        // a) ignore non-symbol characters
        if (!isOpening(c) && !isClosing(c)) {
            continue;
        }

        // b) opening symbol -> push
        if (isOpening(c)) {
            stack.push(c);
        }
        // c) closing symbol
        else if (isClosing(c)) {
            // i) report error if stack is empty
            if (stack.isEmpty()) {
                cout << "Error: closing symbol '" << c
                     << "' found with no matching opening symbol." <<
endl;
                return false;
            }
            // ii) otherwise pop the stack
            char popped = stack.pop();

            // d) if popped symbol does not match, report error
            if (!isMatchingPair(popped, c)) {
                cout << "Error: '" << popped << "' does not match '" <<
c << "'." << endl;
                return false;
            }
        }
    }

    // 3) at end of input, if stack is not empty, report an error
    if (!stack.isEmpty()) {
        cout << "Error: unmatched opening symbol(s) remain." << endl;
        return false;
    }

    return true;
```

```
}

int main() {
    string expressions[] = {
        "(A+B)+(C-D)",
        "((A+B)+(C-D)",
        "((A+B)+[C-D])",
        "((A+B]+[C-D]}"
    };

    for (const string& expr : expressions) {
        cout << "Expression: " << expr << endl;
        bool result = isBalancedArray(expr);
        cout << "Valid? " << (result ? "Y" : "N") << endl;
        cout << "--------------------------------" << endl;
    }

    return 0;
}
```

Expression	Valid?	Output	Analysis
(A+B)+(C-D)	Y	Expression: (A+B)+(C-D) Valid? Y	Every (has a matching) in the correct order
((A+B)+(C-D)	N	Expression: ((A+B)+(C-D) Error: unmatched opening symbol(s) remain. Valid? N --------------------------------	An opening (is never closed, stack isn't empty at the end
((A+B)+[C-D])	Y	Expression: ((A+B)+[C-D]) Valid? Y	Symbols nest and close correctly (,(,[,],,),)
((A+B]+[C-D]}	N	Expression: ((A+B]+[C-D]} Error: '(' does not match ']'. Valid? N	The] after A+B doesn't match the (on top of the stack. Wrong closing symbol

Stacks using List

```cpp
#include <iostream>
#include <string>
using namespace std;

// ---------- STACK USING LINKED LISTS ----------
class Node {
public:
    char data;
    Node* next;
};

class ListStack {
private:
    Node* head;

public:
    ListStack() {
        head = nullptr;
    }

    ~ListStack() {
        while (!isEmpty()) {
            pop();
        }
    }

    bool isEmpty() {
        return head == nullptr;
    }

    void push(char c) {
        Node* newNode = new Node();
        newNode->data = c;
        newNode->next = head;
        head = newNode;
    }

    char pop() {
```

```
        if (isEmpty()) {
            cout << "Stack Underflow." << endl;
            return '\0';
        }
        Node* temp = head;
        char val = temp->data;
        head = head->next;
        delete temp;
        return val;
    }
};

// ---------- SYMBOL HELPERS ----------
bool isOpening(char c) {
    return c == '(' || c == '{' || c == '[';
}

bool isClosing(char c) {
    return c == ')' || c == '}' || c == ']';
}

bool isMatchingPair(char open, char close) {
    return (open == '(' && close == ')') ||
           (open == '{' && close == '}') ||
           (open == '[' && close == ']');
}

// ---------- BALANCE CHECKING ALGORITHM ----------
bool isBalancedList(const string& expr) {
    ListStack stack;

    for (size_t i = 0; i < expr.length(); i++) {
        char c = expr[i];

        // a) ignore non-symbol characters
        if (!isOpening(c) && !isClosing(c)) {
            continue;
        }
```

```
        // b) opening symbol -> push
        if (isOpening(c)) {
            stack.push(c);
        }
        // c) closing symbol
        else if (isClosing(c)) {
            // i) report error if stack is empty
            if (stack.isEmpty()) {
                cout << "Error: closing symbol '" << c
                     << "' found with no matching opening symbol." <<
endl;
                return false;
            }
            // ii) otherwise pop the stack
            char popped = stack.pop();

            // d) if popped symbol does not match, report error
            if (!isMatchingPair(popped, c)) {
                cout << "Error: '" << popped << "' does not match '" <<
c << "'." << endl;
                return false;
            }
        }
    }

    // 3) at end of input, if stack is not empty, report an error
    if (!stack.isEmpty()) {
        cout << "Error: unmatched opening symbol(s) remain." << endl;
        return false;
    }

    return true;
}

int main() {
    string expressions[] = {
        "(A+B)+(C-D)",
```

```
        "((A+B)+(C-D)",
        "((A+B)+[C-D])",
        "((A+B]+[C-D]}"
    };

    for (const string& expr : expressions) {
        cout << "Expression: " << expr << endl;
        bool result = isBalancedList(expr);
        cout << "Valid? " << (result ? "Y" : "N") << endl;
        cout << "-------------------------------" << endl;
    }

    return 0;
}
```

Expression	Valid?	Output	Analysis
(A+B)+(C-D)	Y	Expression: (A+B)+(C-D) Valid? Y	Every (has a matching) in the correct order
((A+B)+(C-D)	N	Expression: ((A+B)+(C-D) Error: unmatched opening symbol(s) remain. Valid? N	An opening (is never closed, stack isn't empty at the end
((A+B)+[C-D])	Y	Expression: ((A+B)+[C-D]) Valid? Y	Symbols nest and close correctly (,(,[,],,),)
((A+B]+[C-D]}	N	Expression: ((A+B]+[C-D]} Error: '(' does not match ']'. Valid? N	The] after A+B doesn't match the (on top of the stack. Wrong closing symbol

How do the different internal representations affect the implementation and usage of the stack?

- Array stack requires you to manually track topIndex, check isFull()/isEmpty(), and manage the array's lifetime (delete[] in the destructor).
- Linked list stack requires you to manage Node creation/deletion for every push/pop, and free any remaining nodes in the destructor to avoid leaks.

Conclusion

In this activity, I've learnt how to initialize stacks using arrays and linked lists and the difference between both of them. I also gained a better understanding of the fundamental operations of a stack, such as push, pop, peek, and checking whether the stack is empty or full. Through implementing both versions, I observed that array-based stacks have a fixed size and are generally simpler to implement, while linked list-based stacks are more flexible because they can dynamically allocate memory as needed.

Additionally, this activity helped me understand how memory management differs between the two implementations. Arrays require a predefined size and can lead to overflow when the stack reaches its capacity, whereas linked lists use pointers to connect nodes, allowing the stack to grow until the available memory is exhausted. Although linked lists provide greater flexibility, they require careful pointer manipulation and proper memory deallocation to avoid memory leaks.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Claude, Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Formatting, Code Assistance, Explanation of Concepts, Idea Generation**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **Asked help for how to do the supplementary activity, fix the code syntaxes, and code generation**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance

with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.